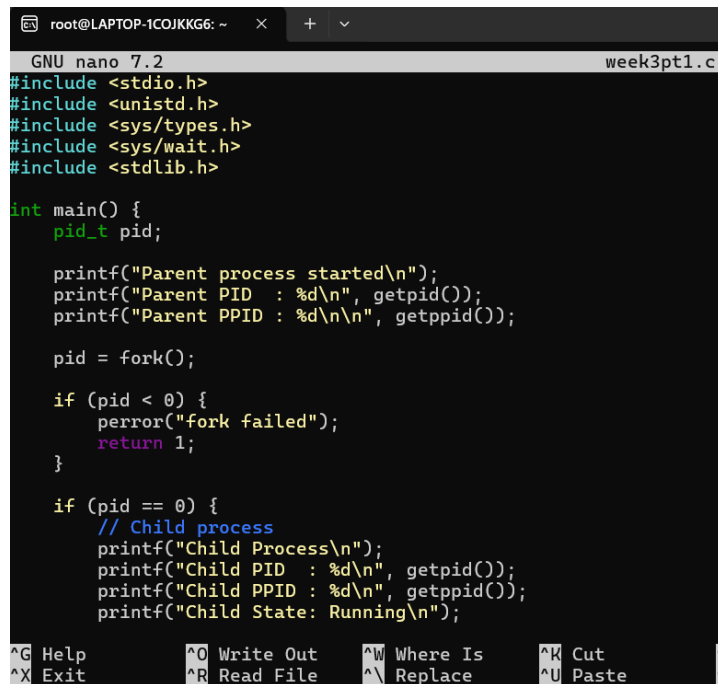


OSSP-PRACTICAL-WEEK3

Task-1: Develop a C program using fork() that creates a parent and child process. Display the Process ID (PID), Parent Process ID (PPID), and process states at different stages of execution.

Source code:



```
GNU nano 7.2 week3pt1.c
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <stdlib.h>

int main() {
    pid_t pid;

    printf("Parent process started\n");
    printf("Parent PID : %d\n", getpid());
    printf("Parent PPID : %d\n\n", getppid());

    pid = fork();

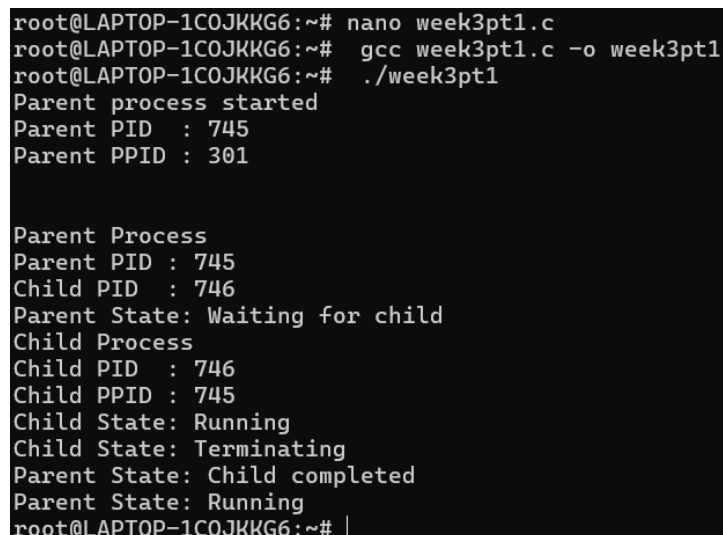
    if (pid < 0) {
        perror("fork failed");
        return 1;
    }

    if (pid == 0) {
        // Child process
        printf("Child Process\n");
        printf("Child PID : %d\n", getpid());
        printf("Child PPID : %d\n", getppid());
        printf("Child State: Running\n");
    }

    // Parent process continues
    printf("Parent Process\n");
    printf("Parent PID : %d\n", getpid());
    printf("Child PID : %d\n", pid);
    printf("Parent State: Waiting for child\n");

    wait(&pid);
    printf("Child Process\n");
    printf("Child PID : %d\n", pid);
    printf("Child PPID : %d\n", getppid());
    printf("Child State: Running\n");
    printf("Child State: Terminating\n");
    printf("Parent State: Child completed\n");
    printf("Parent State: Running\n");
}
```

Output:



```
root@LAPTOP-1COJKKG6:~# nano week3pt1.c
root@LAPTOP-1COJKKG6:~# gcc week3pt1.c -o week3pt1
root@LAPTOP-1COJKKG6:~# ./week3pt1
Parent process started
Parent PID : 745
Parent PPID : 301

Parent Process
Parent PID : 745
Child PID : 746
Parent State: Waiting for child
Child Process
Child PID : 746
Child PPID : 745
Child State: Running
Child State: Terminating
Parent State: Child completed
Parent State: Running
root@LAPTOP-1COJKKG6:~#
```

Observation: I learned how the fork() system call creates a child process from a parent process. I understood how to display and differentiate the PID and PPID of parent and child processes. I also learned how a parent process waits for the child process to complete using wait() and observed the different process states during execution.

Task2: The aim is to observe process state transitions such as Running, Waiting/Sleeping, and Terminated using Linux commands like ps, top, and /proc.

Source code:

```
GNU nano 7.2
#include <stdio.h>
#include <unistd.h>

int main() {
    printf("Process started\n");
    printf("PID : %d\n", getpid());
    printf("PPID : %d\n", getppid());

    printf("Process State: Running\n");

    printf("Process State: Sleeping/Waiting\n");
    sleep(10);

    printf("Process State: Running again\n");

    printf("Process State: Terminating\n");

    return 0;
}
```

Output:

```
root@LAPTOP-1C0JKKG6:~# gcc week3pt2.c -o week3pt2
root@LAPTOP-1C0JKKG6:~# ./week3pt2
Process started
PID : 791
PPID : 301
Process State: Running
Process State: Sleeping/Waiting
Process State: Running again
Process State: Terminating
```

Using command top():

```
top - 03:38:10 up 46 min, 1 user, load average: 0.00, 0.00, 0.00
Tasks: 22 total, 1 running, 21 sleeping, 0 stopped, 0 zombie
%Cpu(s): 0.0 us, 0.0 sy, 0.0 ni,100.0 id, 0.0 wa, 0.0 hi, 0.0 si, 0.0 st
MiB Mem : 7782.6 total, 7195.6 free, 516.6 used, 220.4 buff/cache
MiB Swap: 2048.0 total, 2048.0 free, 0.0 used, 7266.1 avail Mem

  PID USER      PR  NI   VIRT   RES   SHR  S  %CPU  %MEM     TIME+ COMMAND
    1 root        20   0   21592  13220  9796  S   0.0   0.2   0:00.50 systemd
    2 root        20   0    3180   2204   2072  S   0.0   0.0   0:00.00 init-systemd(Ub
    7 root        20   0    3196   2132   2008  S   0.0   0.0   0:00.00 init
   53 root        19  -1   33976  12636  11560  S   0.0   0.2   0:00.18 systemd-journal
  103 root        20   0   25464   6852   5124  S   0.0   0.1   0:00.55 systemd-udev
  112 systemd+   20   0   21344  13184  11056  S   0.0   0.2   0:00.07 systemd-resolve
  120 systemd+   20   0   91036   7988   7020  S   0.0   0.1   0:00.09 systemd-timesyn
  170 root        20   0    4244   2684   2432  S   0.0   0.0   0:00.01 cron
  171 message+   20   0    9528   5364   4668  S   0.0   0.1   0:00.06 dbus-daemon
  188 root        20   0   17980   8852   7808  S   0.0   0.1   0:00.09 systemd-logind
  190 root        20   0 1755848  10780   8404  S   0.0   0.1   0:00.16 wsl-pro-service
  209 syslog     20   0  222516   6068   4676  S   0.0   0.1   0:00.08 rsyslogd
  212 root        20   0    3124   1972   1836  S   0.0   0.0   0:00.00 agetty
  218 root        20   0  107032  23184  13724  S   0.0   0.3   0:00.12 unattended-upgr
  296 root        20   0    3184   1108    980  S   0.0   0.0   0:00.00 SessionLeader
  297 root        20   0    3200   1244   1108  S   0.0   0.0   0:00.14 Relay(301)
  301 root        20   0    6188   5464   3708  S   0.0   0.1   0:00.14 bash
  304 root        20   0    6672   4496   3728  S   0.0   0.1   0:00.00 login
  351 root        20   0   20328  11512   9388  S   0.0   0.1   0:00.04 systemd
  352 root        20   0   21168   3496   1784  S   0.0   0.0   0:00.00 (sd-pam)
  372 root        20   0    6080   5332   3668  S   0.0   0.1   0:00.01 bash
  815 root        20   0    9280   5660   3500  R   0.0   0.1   0:00.02 top
```

Using ps-ef command:

```
root@LAPTOP-1C0JKKG6:~# ps -ef | grep week3pt2
root      835      301  0 03:41 pts/0    00:00:00 grep --color=auto week3pt2
```

Observation: The process was executed and monitored using Linux process-monitoring commands. The top command was used to observe the running process, while ps was used to identify the process and its PID. The experiment showed that a process can change between **Running** and Sleeping/Waiting states during execution. The process finally terminates after completing its execution.